IoT Security & Cybersecurity

Master 2 — Computer Science

Session 3 of 12 — Cryptography II: Advanced Signatures & Certification

Where We Are

Session 2 closed with a single-signer RSA signature and a preview question: how can several signers jointly produce one compact signature, and what other guarantees might a signature scheme provide beyond plain authentication? This session answers that question in full: ring, group, blind, aggregated, and threshold signatures, followed by certification (how public keys are trusted in the first place) and key exchange protocols (how two parties agree on a shared secret over an insecure channel).

1 Ring Signatures

Definition — Ring Signature

Given a set of n public keys $R = \{pk_1, \ldots, pk_n\}$ (the "ring"), a ring signature scheme lets any member i holding the matching private key sk_i produce a signature σ over a message m such that: (1) verification using R and σ succeeds, confirming some member of R signed m; (2) no efficient verifier can determine **which** member i actually signed, even given unlimited computational resources on the ring's public keys (unconditional anonymity).

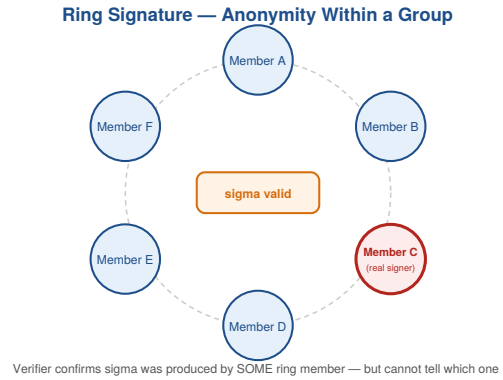


Figure 1: Six possible signers form a ring; a verifier confirms the signature is valid for the ring as a whole, but cannot identify which member actually produced it.

Example — Ring signatures for anonymous IoT incident reporting

Consider an industrial IoT deployment where any one of several field engineers may need to report a safety violation anonymously, without fear of retaliation, while the organization still needs cryptographic proof that the report came from an authorized engineer (not an outsider). A ring signature over the set of all engineers' public keys achieves exactly this: verifiable authenticity of origin, with the specific author hidden within the group.

? Think about it

A ring signature requires no coordination or setup among ring members — a signer can include any public keys they like in the ring, even without those other members' knowledge or consent. Why is this property essential to the whistleblower use case?

✓ Answer

Because requiring the cooperation of the other ring members to form the ring would itself leak information (those members would know a signature involving them was about to be produced, and could potentially identify the actual signer by elimination). Since ring membership can be assembled unilaterally from already-public keys, the true signer never has to reveal their intent to anyone else in the ring.

2 Group Signatures

Definition — Group Signature

A group signature scheme resembles a ring signature but introduces a designated group manager holding a special opening key ok. Any group member can sign anonymously on the group's behalf (verifiers learn only "a group member signed this," as with ring signatures), but the group manager, and only the group manager, can use ok to reveal (open) the true signer's identity if required.

Example — Group signatures vs. ring signatures in practice

The key distinction is accountability. A ring signature offers permanent, unconditional anonymity: no one, ever, can identify the signer. A group signature offers conditional anonymity: ordinary verifiers cannot identify the signer, but a trusted authority can, if misuse is suspected. Vehicle-to-vehicle (V2V) communication standards use group signatures for exactly this reason: a car's broadcast messages must be anonymous to other road users (to protect driver privacy) but traceable by a regulatory authority if a vehicle is found to be broadcasting malicious or malformed traffic.

? Think about it

Give an example.

✓ Answer

The IEEE 1609.2 standard for vehicular ad-hoc networks (VANETs) uses group-signature-like pseudonym certificate schemes precisely so that a misbehaving vehicle can be identified and revoked by a certificate authority, while honest vehicles remain unlinkable to outside observers monitoring the broadcast traffic.

3 Blind Signatures

Definition — Blind Signature

A blind signature scheme lets a signer produce a valid signature over a message without ever seeing the message's actual content. The requester "blinds" m into $m' = \text{Blind}(r, m)$ using a random blinding factor r, obtains a signature $\sigma' = \text{Sign}(sk, m')$ from the signer, then "unblinds" it locally: $\sigma = \text{Unblind}(r, \sigma')$, such that σ is a valid signature on the original m — yet the signer never learned m, and cannot later link σ back to the specific blinded request m'

they signed.

Example — RSA blind signature — the mathematical mechanism

Using RSA public key (n, e): the requester picks a random r coprime to n and sends $m' = m \cdot r^e \bmod n$. The signer, seeing only m', returns $\sigma' = (m')^d \bmod n$. The requester computes $\sigma = \sigma' \cdot r^{-1} \bmod n$. Since $\sigma' = m^d \cdot r^{ed} \bmod n = m^d \cdot r \bmod n$ (because $ed \equiv 1 \pmod{\varphi(n)}$), dividing by r yields exactly $\sigma = m^d \bmod n$, a valid standard RSA signature on the original, never-disclosed m.

? Think about it

An IoT firmware distribution service wants to issue "download tokens" that let a device fetch a firmware update anonymously, without the distribution server being able to correlate which device requested which specific update version — useful for privacy-preserving telemetry opt-outs. How could blind signatures achieve this?

✓ Answer

The device blinds a randomly generated token, sends it for signing, and unblinds the returned signature locally. The distribution server signs a value it cannot read (so it learns nothing about which token it signed), and later, when the device presents the unblinded, validly signed token to download firmware, the server can verify the signature is genuine but cannot link this presentation back to the specific blind-signing request that produced it.

4 Aggregated Signatures

Definition — Aggregate Signature

Given n signers with distinct key pairs (pk_i, sk_i), each producing an individual signature σ_i on a (possibly distinct) message m_i, an aggregate signature scheme provides an algorithm $\text{Aggregate}(\sigma_1, \ldots, \sigma_n) \to \sigma_{\text{agg}}$ that compresses all n signatures into a single, constant-size signature. Verification of σ_{agg} against all n (message, public key) pairs succeeds if and only if every individual signature was valid.

Example — BLS aggregate signatures in sensor networks

BLS (Boneh–Lynn–Shacham) signatures, built on pairing-friendly elliptic curves, support particularly simple aggregation: signatures are single elliptic-curve points, and aggregation is just point addition. A gateway collecting readings from 200 sensors, each individually signing its reading, can compress all 200 signatures into one aggregate signature of the same fixed size as a single signature before forwarding to the cloud — dramatically reducing bandwidth compared to forwarding 200 separate signatures.

? Think about it

Aggregation saves bandwidth, but what does a verifier lose compared to checking 200 individual signatures separately?

✓ Answer

Granular fault localization: if aggregate verification fails, the verifier learns only that at least one of the 200 signatures was invalid, not which one. In a real deployment this trade-off matters — a compromised or malfunctioning sensor's bad reading would force the gateway to discard (or individually re-check) the entire batch rather than simply dropping the one bad reading, unless the protocol is specifically designed with a fallback mechanism for this case.

5 Threshold Signatures

Definition — (t, n)-Threshold Signature

A private signing key sk is split, via a secret-sharing scheme, into n shares $sk_1, \ldots, sk_n$ distributed among n parties, such that any subset of t or more parties can jointly compute a valid signature (each contributing a partial signature that is combined), while any subset of fewer than t parties learns nothing usable about sk and cannot forge a signature.

Example — Threshold signatures for distributed IoT key management

A fleet of IoT gateways collectively authorizes firmware pushes to the devices under their management. Rather than trusting one gateway with the full signing key (a single point of compromise), a $(3, 5)$-threshold scheme across five gateways ensures that compromising any two gateways gives an attacker nothing, while the system tolerates up to two gateways being offline or faulty and still functions normally.

5.1 The Underlying Mechanism: Shamir's Secret Sharing

Threshold signature schemes typically build on a general primitive for splitting any secret value among n parties.

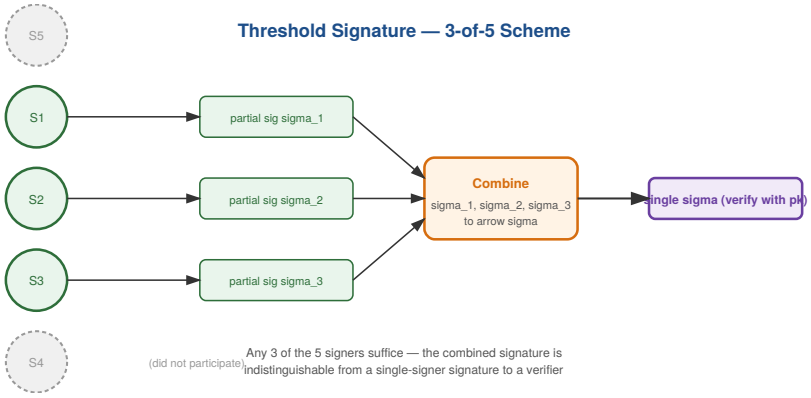


Figure 2: A 3-of-5 threshold scheme: any 3 of the 5 signers suffice to jointly produce a single combined signature, indistinguishable from a single-signer signature to an external verifier.

Definition — Shamir's (t, n)-Secret Sharing

To split secret $s \in \mathbb{Z}_q$ among n parties with threshold t: construct a random polynomial $f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}$ of degree $t-1$ over $\mathbb{Z}_q$, with $f(0) = s$. Party i receives share $(i, f(i))$. Any t shares uniquely determine the degree-$(t-1)$ polynomial via Lagrange interpolation, recovering $f(0) = s$; any $t-1$ or fewer shares leave s information-theoretically undetermined, since infinitely many degree-$(t-1)$ polynomials pass through $t-1$ points.

Example — Why $t-1$ shares reveal nothing, not just "not enough"

This is a stronger guarantee than most cryptographic hardness assumptions: with only $t-1$ shares, every possible value of $s \in \mathbb{Z}_q$ remains equally consistent with the observed shares — there exists a valid degree-$(t-1)$ polynomial through those $t-1$ points for **any** choice of $f(0)$. An attacker with unlimited computational power, not just limited time, still learns literally nothing about s from $t-1$ shares. This is called information-theoretic (rather than merely computational) security.

? Think about it

A $(3, 5)$ threshold scheme splits a signing key across five IoT gateways. An attacker breaches two gateways and steals their key shares. What can the attacker conclude about the actual signing key?

✓ Answer

Nothing whatsoever about the key's value — with only 2 shares out of a required threshold of 3, Shamir's construction guarantees information-theoretic secrecy: every possible signing key remains equally likely given the two stolen shares. The attacker would need to compromise a third gateway before gaining any advantage at all, and even then would need the actual live cooperation of a third share-holder, not just the raw share value in isolation.

6 Lightweight Certificates for Constrained Devices

Standard X.509 certificates, used across the web, are verbose (often 1–2 KB) and expensive to parse on constrained hardware. IoT-specific profiles reduce this overhead.

Example — C509 and IEEE 802.1AR compact certificates

The IETF's C509 format re-encodes X.509 certificate fields using CBOR (Concise Binary Object Representation) instead of ASN.1 DER, achieving certificates roughly one-third the size of equivalent X.509 encodings, with faster parsing on constrained CPUs. IEEE 802.1AR similarly defines a compact "Secure Device Identity" (DevID) certificate format specifically for embedding a verifiable, manufacturer-issued identity directly into IoT hardware at manufacturing time.

? Think about it

Give an example from the real world.

✓ Answer

Matter, the smart-home interoperability standard backed by major manufacturers, requires every certified device to carry a Device Attestation Certificate at manufacturing time, built on a compact certificate chain rooted in a Certification Authority operated by the Connectivity Standards Alliance — allowing any Matter- compatible hub to cryptographically verify a new device's manufacturer and model before admitting it to the network, all within the tight code-size budget of a constrained smart-home device.

7 Closing the Loop: How Multi-Signatures Actually Combine

Session 2 ended by asking how several signers can combine partial signatures into one compact signature without any signer revealing their private key to the others. Schnorr signatures answer this particularly cleanly, and the same structure underlies modern constructions such as MuSig.

Definition — Schnorr Signature

Given a group generator g of prime order q and private key x with public key $pk = g^x$: to sign message m, the signer picks random nonce k, computes commitment $R = g^k$, challenge $c = H(R\|pk\|m)$, and response $s = k + cx \bmod q$. The signature is (R, s). Verification checks $g^s = R \cdot pk^c$.

Example — Two-round multi-signature combination

For n signers to jointly sign under Schnorr: round 1, every signer i generates a nonce k_i, computes $R_i = g^{k_i}$, and broadcasts R_i to the others; each party locally combines $R = \prod_i R_i$ and the combined public key $pk_{\text{agg}} = \prod_i pk_i$. Round 2, each signer computes their own partial response $s_i = k_i + c \cdot x_i \bmod q$ using the shared challenge $c = H(R\|pk_{\text{agg}}\|m)$, and broadcasts s_i. Anyone can then combine $s = \sum_i s_i \bmod q$, producing a single signature (R, s) that verifies against pk_{agg} as though one signer had produced it, yet no single signer's private key x_i was ever transmitted or reconstructed by anyone.

? Think about it

Why does the protocol require two rounds of communication rather than one — why can't each signer just directly send their final partial response s_i in a single round?

✓ Answer

Because the challenge c used to compute each s_i depends on the combined commitment R, which itself depends on every signer's individual R_i. If a signer could see all other signers' R_i values before choosing their own, a malicious signer could adaptively choose R_i to bias the final combined signature (a real attack known against naive one-round variants); a strict

two-round structure, where commitments are fixed before any response is revealed, prevents this manipulation.

8 Comparing the Signature Family

Scheme	Core property	Typical IoT use case
Single (RSA/ECDSA)	One signer, one verifiable identity	Signed firmware updates
Ring	Unconditional anonymity within a set	Anonymous incident reporting
Group	Conditional (manager-revocable) anonymity	V2V / VANET pseudonymous broadcasts
Blind	Signer does not see the message content	Anonymous firmware download tokens
Aggregate	Many signatures compress to one	Bandwidth-efficient sensor telemetry batches
Threshold	t-of-n fault-tolerant joint signing	Distributed gateway key management

Notice that these are not mutually exclusive design axes. A production system can, in principle, combine several: a threshold group signature, for instance, would let any t of n group members jointly produce a signature that is anonymous to ordinary verifiers but traceable by the group manager — exactly the kind of composite construction increasingly used in permissioned blockchain systems for IoT device identity, where accountability, fault tolerance, and privacy are all required simultaneously.

Every signature scheme discussed so far assumes the verifier already knows which public key genuinely belongs to which party. This assumption itself requires a trust infrastructure.

Definition — Digital Certificate

A digital certificate is a data structure binding a public key pk to an identity (e.g., a device serial number or a domain name), digitally signed by a trusted Certificate Authority (CA) using the CA's own private key sk_{CA}:

$$\mathrm{Cert} = (\mathrm{identity}, pk, \mathrm{Sign}(sk_{\mathrm{CA}}, \mathrm{identity} \| pk)).$$

A verifier who trusts the CA's public key pk_{CA} can verify the certificate's signature and thereby trust that pk genuinely belongs to the named identity.

Example — Certificate chains and the root of trust

In practice, a single CA rarely signs every device certificate directly. Instead, a root CA signs intermediate CA certificates, which in turn sign device (leaf) certificates, forming a certificate chain. A verifier need only pre-trust the small set of root CA public keys, then verify the entire chain of signatures down to the leaf certificate. IoT devices commonly ship with a small, fixed set of trusted root certificates burned into firmware at manufacturing time.

? Think about it

If a device's firmware trusts a root CA certificate that is never updated over the device's ten-year operational lifetime, what security risk does this create?

✓ Answer

Key rot and algorithm obsolescence: cryptographic standards and recommended key sizes evolve (an algorithm considered secure today may be broken within a decade), and a compromised or expired root CA that the device can never stop trusting represents a permanent, unpatchable trust anchor. This is why long-lived IoT deployments increasingly require a secure, authenticated mechanism for updating trusted root certificates remotely, not just device firmware.

9 Certificate Revocation

A certificate, once issued, may need to be invalidated before its stated expiry — a device is decommissioned, a private key is suspected compromised, or an issuing CA is itself breached.

Example — CRL vs. OCSP

A Certificate Revocation List (CRL) is a periodically published, signed list of revoked certificate serial numbers that a verifier downloads and checks locally. The Online Certificate Status Protocol (OCSP) instead lets a verifier query a live responder for the real-time status of one specific certificate. CRLs scale poorly as revocation lists grow (large downloads) but work offline once fetched; OCSP is lightweight per query but requires network connectivity at verification time and can leak which certificates a given verifier is checking.

? Think about it

An IoT sensor deployed in a remote location with only intermittent network connectivity needs to validate a peer's certificate. Which revocation-checking approach is more practical, CRL or OCSP, and what is the security trade-off of choosing it?

✓ Answer

CRL is more practical: it can be fetched during the rare windows of connectivity and consulted entirely offline afterward. The trade-off is staleness — a CRL fetched weeks ago will not reflect a certificate revoked yesterday, so the sensor risks trusting a certificate that has since been revoked, a risk that OCSP's real-time query would avoid if connectivity allowed it.

Example — OCSP stapling as a middle ground

OCSP stapling lets the certificate holder itself periodically fetch a signed, timestamped OCSP response from the CA and attach ("staple") it directly to its own certificate presentation. A verifier then gets a reasonably fresh revocation proof without making its own network query at connection time — combining CRL-like offline verification with OCSP-like recency, at the cost of the certificate holder needing intermittent connectivity to refresh its stapled response.

10 Key Exchange Protocols

Definition — Diffie–Hellman Key Exchange

Given a public generator g of a cyclic group of prime order q: party A picks random $a \in \mathbb{Z}_q$ and sends g^a; party B picks random $b \in \mathbb{Z}_q$ and sends g^b. Both compute the shared secret g^{ab}: A computes $(g^b)^a$, B computes $(g^a)^b$. Security rests on the Diffie–Hellman assumption: an eavesdropper observing only g, g^a, and g^b cannot feasibly compute g^{ab}.

Example — ECDHE and forward secrecy

Elliptic-curve Diffie–Hellman Ephemeral (ECDHE) generates a fresh, temporary key pair for every single session rather than reusing a long-term key. This provides forward secrecy: even if a device's long-term private key is later compromised, past recorded sessions — protected by ephemeral keys that were discarded immediately after use — remain unreadable, because the attacker never captured the ephemeral secrets needed to recompute those past session keys.

? Think about it

Give an example from the real world.

✓ Answer

An attacker who passively records years of encrypted IoT gateway-to-cloud traffic, then years later physically extracts the gateway's long-term private key, still cannot decrypt any of the recorded historical traffic if the sessions used ECDHE — this is precisely why TLS 1.3 mandates ephemeral key exchange and removed support for static (non-forward-secret) RSA key exchange entirely.

10.1 Authenticating the Exchange: Station-to-Station

Plain Diffie–Hellman, as defined above, provides no authentication on its own. The Station-to-Station (STS) protocol shows one clean way to combine key exchange with signatures.

Example — Station-to-Station protocol

A sends g^a. B computes the shared secret g^{ab}, then sends back g^b together with a signature over the pair (g^a, g^b) using B's certified long-term key, plus a symmetric encryption of that signature under g^{ab} (binding the signature to the session). A verifies B's signature and certificate, then replies with its own signature over (g^b, g^a), similarly encrypted. Both parties now share an authenticated secret g^{ab}, with each having cryptographic proof of the other's identity.

? Think about it

Diffie–Hellman as described provides no authentication — an active man-in-the-middle could intercept and replace both g^a and g^b with their own values. How do the certification mechanisms from Section 4 fix this?

✓ Answer

By binding the exchanged Diffie–Hellman values to signatures verifiable via the parties' certified public keys: in a protocol like TLS, the server signs its ephemeral g^b value using the long-term private key certified by its CA-issued certificate. A man-in-the-middle without that private key cannot produce a valid signature over a substituted value, so the client detects and rejects the tampered exchange — combining certification (Section 4) with key exchange (this section)

into the authenticated key agreement used throughout real-world protocols.

Session Summary

This session completed the signature family opened in Session 2: ring signatures (unconditional anonymity), group signatures (conditional, revocable anonymity), blind signatures (signing without seeing), aggregate signatures (bandwidth-efficient batch verification), and threshold signatures (fault-tolerant distributed signing, built on Shamir's information-theoretically secure secret sharing). It then connected signatures to trust infrastructure via certification, revocation, and lightweight IoT certificate formats, and closed with Diffie–Hellman key exchange, forward secrecy, and the Station-to-Station protocol for authenticated key agreement. Session 4 turns to a different mathematical territory: homomorphic encryption and zero-knowledge proofs.